A Validation Layer for AI-to-Unix Execution

AIShell-Gate as an AI Execution Gateway: a deterministic policy layer mediating between probabilistic AI systems and deterministic infrastructure.

aishell-gate-policy 1.09 · aishell-gate-exec 0.42.0 · aishell-gate-mcp 1.0

Sean T. Gilley · AIShell Labs LLC · Winston-Salem NC USA
www.aishellgate.com · www.aishell.org · info@aishell.org · sgilley@aishellgate.com

Author of The Shape of Intent: Steering Artificial Intelligence in Cognitive Collaboration

Copyright © 2026 AIShell Labs LLC. All Rights Reserved. Use of this software requires a valid license.

SECURITY POSITIONING

AIShell-Gate is not a standalone security product and does not replace operating system, network, or organizational security controls. It is a risk-reduction layer designed to formalize and evaluate AI-initiated actions before execution. AIShell-Gate is hardened and engineered to operate as a controlled execution boundary, but it is intended to function as one component within a broader security architecture. It should be deployed alongside standard access controls, system permissions, monitoring, and established security practices.

ABSTRACT

Large language models are increasingly used to generate Unix shell commands in operational and development environments. Their output is probabilistic; Unix execution is deterministic and irreversible. This structural mismatch creates a missing layer between AI suggestion and system execution. AIShell-Gate addresses that gap with a deterministic policy engine and execution gateway implemented in portable C with no external dependencies. Every proposed command is evaluated against a configurable, layered policy before execution; the policy engine and executor are separated into distinct processes across a hard OS boundary so that neither can subvert the other. The system enforces argument-level access controls, path restrictions, network target rules, and risk-scored confirmation escalation, and produces a tamper-evident HMAC-SHA256 audit chain of every decision. AIShell-Gate serves two distinct user populations from a single unified policy engine: AI agents submitting JSON execution plans, and human operators working interactively at a terminal. The same gate, the same policy, and the same audit chain apply to both. For human operators, the system functions simultaneously as a safety net, a teaching tool — every flag assessment carries documented reasoning explaining *why* a command is categorized as it is, not merely *what* the decision is — and a disciplined workflow enforcing confirmation gates and audit trail on operator-generated commands. A novel threat class addressed by version 1.02 is *Model Sub-Delegation* — an AI agent consulting a second inference system to influence a policy decision without the operator's knowledge. The specific mechanism is *Self-Referential API Access*: the agent contacts inference endpoints — local, remote, or its own model's public API — from within a gated execution session. The practice is not inherently prohibited; AIShell-Gate's gate ensures the operator is aware it is occurring and explicitly approves it, converting an invisible action into a conscious decision. Version 1.0 Beta covers the local-inference variant through the 108-entry port catalog and loopback-alias normalization; a hostname catalog extension covering remote and same-model variants is architecturally adjacent and planned for a subsequent release. The architecture is intended to occupy the specific gap between unrestricted AI shell access and no AI integration at all, providing a practical, auditable execution boundary for organizations introducing AI into Unix workflows. Version 1.02 introduces an MCP (Model Context Protocol) server — *aishell-*

gate-mcp — that exposes two tools to AI coding environments such as Claude Code and Cursor: *evaluate_plan* for policy-gated plan inspection without execution, and *execute_plan* for live execution. Every MCP tool response includes a protocol version block, making the full wire interface versioned and forward-compatible. The flag catalog has expanded to 2,088 entries covering ansible, ansible-playbook, az (Azure CLI), extended kubectl, terraform, and helm flags, kernel operations (insmod, kexec, rmmod), disk partitioning tools, debugfs, socat, nftables, and the full LVM suite. Network policy enforcement has been formalized: *net_default_deny* — decoupled from command default-deny — requires explicit allow rules for any detected network target in ops_safe, read_only, and dev_sandbox presets, while CI presets allow unrestricted network access for package registry and artifact store access. New operator tools include `--dry-run-json`, which produces a machine-readable JSON document describing what a plan would execute without executing it, and `--test-plan`, a policy testing mode that evaluates a JSON file of test cases against the active policy and reports PASS/FAIL, suitable for committing alongside policy files and running in CI. This paper introduces a formal glossary of terminology coined for the AI execution security domain, including *Model Sub-Delegation*, *Self-Referential API Access*, *Command-Coded Confirmation*, *Zero-Effect Fail-Closed*, *Declared and Injected Source Identity*, *Epistemic Honesty*, *Execution Posture*, *Net Default-Deny*, and *Broker Seam*. See §18.

CONTENTS

1. The Problem
2. Where AIShell-Gate Sits in the Stack
3. What AIShell-Gate Is
4. Internal Architecture
5. Policy Layer Model
6. Intended Usage Model
7. Example Decisions
8. Example of Full Integration
9. What AIShell-Gate Is Not
10. Threat Model
11. Model Sub-Delegation & Self-Referential API Access
12. Source Code: Security in Practice
13. Design Principles
14. Who Needs This
15. Current Implementation Features
16. Audit and Accountability
17. MCP Integration
18. MCP Integration
19. Status
20. Terminology Glossary
21. References

01 The Problem

Large language models are increasingly used to generate shell commands. They are good at it. They are also probabilistic.

Unix execution, by contrast, is deterministic and irreversible. A single misplaced flag, path, or wildcard can delete data, overwrite files, or alter system state permanently.

This creates a structural mismatch: AI output is statistical. Unix execution is exact.[1] The distinction has a practical consequence that Gilley articulates in the broader context of AI collaboration: brought intent, AI produces leverage; brought confusion, it generates the appearance of progress.[10] In the Unix execution context, the appearance of progress is a particularly dangerous failure mode — a command that looks correct executes with the same irreversibility as one that is correct.

When AI-generated commands are introduced into production workflows, there is a missing layer between suggestion and execution. That missing layer is policy. AIShell-Gate exists to provide that layer.

Trust in AI systems cannot be achieved by attempting to make probabilistic models perfectly reliable. Instead, trust emerges from architecture: deterministic boundaries that govern how AI-generated actions interact with real systems.[2] AIShell-Gate addresses one such boundary — Unix command execution — converting probabilistic AI suggestions into controlled, auditable system operations.

02 Where AIShell-Gate Sits in the Stack

AIShell-Gate is not an AI system and not a shell. It is a policy boundary between AI-generated commands and Unix execution.

```
AI Agent (Claude Code / Cursor) AI Agent (direct) Human Operator │ MCP tool call │ JSON plan │
interactive ▼ ▼ ▼ aishell-gate-mcp ◄ MCP server (stdio) aishell-gate-exec ◄ confirmation + execution
wrapper evaluate_plan / execute_plan │ │ builds JSON plan envelope │
└──────────────────────────────────────────────┘ │ JSON plan { protocol, goal, source, strategy,
actions } ▼ aishell-gate-policy ◄ deterministic policy evaluation normalise → match rules → risk
score → JSON decision │ ALLOW / DENY · confirm level · risk score ▼ execve(absolute_path,
validated_argv, safe_environment) (no shell · no PATH inheritance · no env injection)
```

LAYER	RESPONSIBILITY
AI agent	Generate candidate actions
aishell-gate-mcp	MCP server — translates tool calls to JSON plan envelopes; exposes evaluate_plan and execute_plan tools to Claude Code, Cursor, and compatible environments
aishell-gate-exec	Handle confirmation and perform execution; submit each action to the policy engine over a hard OS boundary
aishell-gate-policy	Evaluate commands against deterministic policy; never executes anything
Operating system	Run the validated command via execve()

AIShell-Gate deliberately sits between the probabilistic layer and the deterministic layer. This boundary ensures that AI output is never executed directly, every command is evaluated by policy, unsafe commands are rejected before execution, and human confirmation can be required for risky actions.

The result is a controlled execution path:

```
AI suggestion → policy evaluation → optional human confirmation → execution
```

rather than the far more common pattern:

```
AI suggestion → direct shell execution
```

03 What AIShell-Gate Is

AIShell-Gate is a deterministic command validation engine. It evaluates a proposed shell command against a declared policy and returns exactly one of two decisions:

- **ALLOW** — command is permitted (subject to a confirmation level).
- **DENY** — command is rejected; execution must not occur.

Every ALLOW decision also carries a confirmation level:

LEVEL	MEANING
`none`	No confirmation required; proceed immediately.
`plan`	Show the plan before executing; human review suggested.
`action`	Explicit per-command human approval required.
`typed`	Human must type a confirmation code derived from the validated command.

Confirmation levels are declared by policy rules and escalated automatically by risk score. A command scoring above 40, 70, or 90 has its level raised to `plan`, `action`, or `typed` respectively, regardless of what the matching rule says. Levels are only ever raised, never lowered.

Risk scoring works in two steps. First, `risk_classify()` assigns a base score from a built-in command catalog — `ls` scores 0, `cp` scores 15, `rm` scores 80, `dd` and `mkfs` score 95 and 98 respectively. Modifiers are then added based on argument structure: +10 if a recursive flag is present on a destructive command, +10 if `--force` is combined with `rm` or `mv`, +15 if any argument targets a system path, +10 if `curl` or `wget` is called with a URL. The blast radius field is set from the same pass: *system* if a root or system path is targeted, *tree* if recursive, *single* otherwise. Second, `risk_apply_confirmation()` applies the score thresholds strictly monotonically — it can only raise the confirmation level, never lower it. Commands not in the catalog start at score 0 and confirmation `none`; they are handled by policy rules, not catalog scoring. All risk fields appear in the JSON output and the audit log so every confirmation escalation is traceable to its cause.

Beyond policy enforcement, this confirmation escalation mechanism functions as an operational

interlock: higher-risk actions trigger progressively stronger confirmation requirements so that irreversible or high-blast-radius commands cannot pass from AI suggestion to execution without deliberate human review. The interlock is structural — it cannot be skipped by the executor, disabled by the AI agent, or overridden by a policy layer below the risk threshold.

AIShell-Gate does not execute commands. It does not attempt to interpret intent. It does not rely on AI to judge risk. It enforces declared rules in a transparent, deterministic way. It is a policy gate between AI output and Unix execution.

04 Internal Architecture: Two Components, One Hard Boundary

The implementation spans two C programs. Their separation is not a convenience — it is the core security property of the system. Conceptually, AIShell-Gate functions as a reference monitor for AI-generated shell commands: a small, trustworthy component that interposes between a subject (the AI agent) and an object (Unix command execution), enforcing complete mediation between probabilistic AI output and deterministic execution.[3]

REFERENCE MONITOR CONCEPT	AISHELL-GATE EQUIVALENT
Subject	AI agent
Object	Unix command execution
Policy engine	aishell-gate-policy
Enforcement	aishell-gate-exec

The separation between the policy engine and the executor also reflects the classic operating-systems distinction between policy and mechanism: the policy engine decides whether an action is permitted; the executor performs the action once permitted. Neither component reaches across that boundary.[4]

`aishell-gate-policy` is the policy engine. It receives a proposed command, normalizes it, evaluates it against the loaded policy stack, computes a risk score, and emits a structured JSON decision. It has no ability to execute anything. It does not spawn processes. It does not open network connections. Its only output is the decision record.

`aishell-gate-exec` is the executor. It accepts a JSON plan from an AI agent or caller, submits each action to the policy engine as a child process, reads the JSON evaluation response back over a pipe, collects human confirmation where required, and calls `execve()` with the validated argv from the response. Critically, it contains no policy logic whatsoever. It cannot approve or deny a command. Every execution decision is made by the policy engine in a separate process, across a hard OS process boundary.

`aishell-confirm` is the remote confirmation relay. In remote deployments where an AI agent connects over SSH, the agent has no terminal and cannot display confirmation prompts. `aishell-confirm` is run by a human operator in a separate SSH session; it receives the confirmation request from the executor over a named FIFO, displays the full context — goal, command, risk score, challenge code — on the operator's terminal, and relays the operator's response back to the executor. The AI's communication channel sees only the final ALLOW or DENY; the confirmation itself is structurally inaccessible to the agent.

That boundary matters. If the executor is compromised — through a bug, a malicious plan, or a hostile environment — it still cannot grant itself permission to run a command that the policy engine has denied, because permission is not a variable inside the executor. It is the output of an independent process that the executor has no ability to modify.

```
AI Agent │ ▼ JSON plan { goal, source, strategy, actions: [ {cmd: ...}, ... ] } aishell-gate-exec
(no policy logic) forks policy engine as child process for each action reads JSON decision back over
pipe │ ▼ aishell-gate-policy (policy engine, separate process) normalise → match rules → risk score
→ JSON decision exits; result returned over pipe only │ ◄──┘ collects human confirmation if
required │ ▼ execve(absolute_path, validated_argv, safe_environment) (no shell, no PATH inheritance,
no env injection)
```

> **THE SPECIFIC SECURITY PROBLEM**
>
> *Classic reference monitors protect users from other users. AIShell-Gate protects a deterministic system from a probabilistic agent. That is the specific security problem AI-to-Unix execution introduces, and it is the problem this architecture is designed to solve.*

05 Policy Layer Model

Policy is not a single file. It is a stack of three layers applied in order: base, project, and user. Each layer can add rules, replace rule sets entirely, or restrict what earlier layers permitted. A deny at any layer is final — later layers cannot promote it back to allow.

The **base layer** is the organizational baseline. It defines the floor: what commands are never permitted regardless of context, which paths are always protected, and which network targets are always blocked. The built-in default base policy denies recursive deletions, writes to system paths, sudo, and access to cloud metadata endpoints (`169.254.169.254` and `metadata.google.internal`).

The **project layer** extends the base for a specific workflow. A git-focused project layer might allow `git status`, `git diff`, and test runners within the project directory without requiring confirmation, while still denying anything outside that scope.

The **user layer** adds personal preferences on top of the project layer — additional allowed commands, preferred confirmation thresholds, or personal directory permissions. It cannot expand beyond what the base and project layers permit.

Presets (`--policy-preset ops_safe`, `--policy-preset dev_sandbox`, etc.) are named configurations that replace the builtin layer's command allow/deny lists while preserving argument, path, and network rules. Each preset defines a complete *Execution Posture* — the full risk and permission profile for a specific workflow type. A preset is a starting posture; operators can then provide override files for any layer to tune it for their environment.

This model means that an organization can harden the base layer once and trust that no project or user configuration can undermine it. Policy is auditable at every layer: the JSON decision record names the layer that matched each action, so an audit trail always shows not just what was allowed or denied but which policy level made that call.

NETWORK DEFAULT-DENY (`NET_DEFAULT_DENY`)

Network policy enforcement is governed by a separate flag — `net_default_deny` — decoupled from the command default-deny gate. When true (the default for `ops_safe`, `read_only`, and `dev_sandbox` presets), any command with a detected network target — a URL, `host:port` argument, or parseable address — must have an explicit `net_rules` allow entry for every target. Targets with no matching allow rule are denied, mirroring the command policy model: unknown is denied, not warned. CI presets (`ci_build`, `ci_deploy`, `ci_admin`) set `net_default_deny` to false, recognising that build pipelines legitimately access package registries, artifact stores, and deployment endpoints that cannot be enumerated in advance. The flag can be set in any policy override file, allowing operators to configure network posture per project. This decoupling is intentional: a CI pipeline may need unrestricted network access while still enforcing the full command allow-list (which remains governed by the unchanged command default-deny).

The built-in network deny rules (cloud metadata endpoints, IPv6 loopback, CGNAT/Tailscale range) remain unconditional regardless of `net_default_deny`. No allow rule in any policy layer can override them.

06 Intended Usage Model

In normal use, an AI agent assembles a plan as a JSON document describing its goal, the source identity (the `source` field in the plan, an *Injected Source Identity* supplied by the agent itself), an execution strategy (`fail_fast` or `best_effort`), and the list of commands to run. For deployments requiring authoritative source attribution, the `--source` flag sets a *Declared Source Identity* at the CLI level that overrides any agent-supplied value — an agent cannot misrepresent itself as human when the deployer has declared the session as AI-sourced. That plan is passed to `aishell-gate-exec`, which handles evaluation, confirmation, and execution as a unit. The plan envelope carries a protocol version block — `"protocol": {"name": "aishell-gate-exec-input", "version": "1.0"}` — identifying the interface version so that future schema changes can be detected rather than silently misinterpreted.

AI agents using Claude Code, Cursor, or any MCP-compatible environment can interact with AIShell-Gate through `aishell-gate-mcp` without constructing envelopes manually. The agent calls `evaluate_plan` to inspect a plan without executing it, then calls `execute_plan` to commit. The MCP server handles envelope construction, protocol versioning, and result parsing. See §17.

INTEGRATION PATTERNS

Three patterns cover all deployment configurations. **Pattern A1 (MCP — local gate)** is the path for Claude Code and Cursor users: the MCP server runs locally as a stdio subprocess; the AI calls `evaluate_plan` and `execute_plan` directly as tools; no pipeline script or SSH configuration is required. **Pattern A2 (HTTPS out — pipeline script)** applies when the AI model is called via a script rather than an MCP-compatible environment: the script calls the model over HTTPS, receives a JSON plan in response, and pipes it to `aishell-gate-exec` on stdin; the gate itself makes no network calls. **Pattern B (SSH in — remote agent)** is the production deployment pattern: the gate runs on the target machine; an AI agent on a remote host has an SSH key whose `authorized_keys` entry specifies `aishell-gate` as the forced command; the agent connects and delivers its JSON plan on stdin and can do nothing else. All three patterns produce the same result at the gate: a JSON plan reaching `aishell-gate-exec`, evaluated by the same policy engine, audited by the same chain.

CONSTRAINED ACCOUNT HARDENING

In Pattern B deployments, constrained account hardening adds a further OS-level enforcement layer. The AI agent's Unix account is given no login shell, no writable home directory, and — critically — no `PATH` and no access to any binary except `aishell-gate-exec`. Policy governs what commands are permitted; the operating system enforces what binaries are reachable. This converts the policy gate from a software boundary into a hard wall: even if the policy engine were compromised, the agent account has no execution surface outside the gate. The `--safe-path` flag on the executor and the INSTALL path C procedure in the distribution configure this pattern.

```
/* Example AI agent plan (input to aishell-gate-exec) */
{
  "protocol": { "name": "aishell-gate-exec-input", "version": "1.0" },
  "goal":     "update project dependencies and run tests",
  "source":   "ai",
  "strategy": "fail_fast",
  "actions": [
    { "type": "command", "cmd": "git pull"      },
    { "type": "command", "cmd": "npm install"   },
    { "type": "command", "cmd": "npm test"      }
  ]
}
```

The executor submits each command to the policy engine in sequence. For each, it receives a JSON decision containing the overall allow/deny, the confirmation level, the matched rule and layer, the validated argv array, and the risk score. The executor never passes the raw command string to a shell. It passes only the validated argv directly to `execve()`.

The policy engine can also be called directly from the command line for single-command evaluation, scripting, or policy testing. The `--json` flag produces machine-readable output. The trace output (Input → Normalized → Matched Rule → Decision → Reason) makes every decision explainable without needing to inspect the policy files.

As of v1.0, AIShell-Gate serves two distinct user populations simultaneously. **AI agents** submit JSON plans as described above. **Human operators** can run either binary interactively at a terminal. `aishell-gate-policy` interactive mode is a purely educational surface: it assesses commands, explains every flag with its documented reasoning, and emits a gateway echo line showing how to route the command through the executor. It never executes. `aishell-gate-exec` interactive mode is the execution surface: it enforces confirmation gates and runs commands with a full audit trail. For the human operator, AIShell-Gate functions simultaneously as a safety net — catching dangerous commands before execution regardless of who typed them — a teaching tool — the 2,088-entry flag catalog delivers documented reasoning on why each flag raises risk, at the point of use, in plain language — and a disciplined workflow providing compliance-grade audit accountability for human shell activity. A junior operator who types an unfamiliar flag learns not just that it was flagged, but specifically why. An experienced operator gets a disciplined daily workflow with gate enforcement and tamper-evident audit throughout. Both populations operate under the same policy file with no additional configuration.

07 Example Decisions

EXAMPLE 1: DENIAL

AI proposes: `rm -rf /var/log/*`

Decision: DENY
Reason: recursive deletion outside allowed paths
Matched rule: block_recursive_system_paths
No execution occurs.

EXAMPLE 2: SAFE OPERATION

AI proposes: `ls -la /home/user/project`

Decision: ALLOW
Confirm: none
Reason: read-only command within permitted directory
Execution layer proceeds without human intervention.

EXAMPLE 3: HIGH-RISK OPERATION

AI proposes: `cp file.txt /etc/config/`

Decision: ALLOW
Confirm: action
Reason: write operation to protected path; explicit confirmation required
The execution layer pauses and requires human approval.

08 Example of Full Integration

The key discipline in any integration: the executor never receives the raw command string for shell evaluation. It reads the validated argv array from the policy engine's JSON output and passes it directly to `execve()`. This eliminates the entire class of injection vulnerabilities that shell evaluation introduces.

```
/* Pseudocode — conceptual integration flow */
result   = aishell_gate_policy("--json", "--policy-preset", "ops_safe", command)
decision = result["overall_decision"]          /* "allow" or "deny" */
confirm  = result["actions"][0]["confirm"]     /* "none"|"plan"|"action"|"typed" */
argv     = result["actions"][0]["argv"]        /* validated argument array */

if decision != "allow":
    log("blocked: " + result["actions"][0]["reason"])
    exit(1)

if confirm == "typed":
    phrase     = derive_challenge(argv)
    user_input = prompt("Type to confirm: " + phrase)
    if user_input != phrase: exit(1)
elif confirm == "action":
    if not prompt_yes_no("Approve: " + join(argv)): exit(1)

execve(argv[0], argv, safe_environment)
```

09 What AIShell-Gate Is Not

AIShell-Gate occupies a specific position in the security landscape and is not a substitute for adjacent mechanisms. Understanding where it sits matters for deployment decisions.

IT IS NOT A SANDBOX — BUT IT IS SANDBOX-AWARE

A sandbox constrains what a running process can do through kernel mechanisms such as seccomp, namespaces, chroot, or containers. AIShell-Gate operates before execution: it decides whether execution should begin at all. These approaches are complementary. The `--sandbox` flag accepts five modes — `none`, `cwd_jail`, `chroot`, `container`, `userns` — and the `--limit-cpu`, `--limit-as-mb`, and `--limit-wall-ms` flags carry resource budgets. These settings are advisory hints for a wrapping executor. The one active exception is `cwd_jail`: when a jail root is configured, the policy engine enforces path containment during evaluation. For the other sandbox modes, the policy engine records the intended mode in the JSON decision and audit log so that a wrapping executor can apply kernel enforcement accordingly.

IT IS NOT SUDOERS — IT OPERATES ON A DIFFERENT AXIS

`sudoers` answers one question: can this user run this command as this other user? It makes a binary identity-based decision at the command level. AIShell-Gate answers a different question: does this specific invocation — these arguments, this path, this network target, at this time, from this session context — satisfy declared policy? `sudoers` can permit `git` for a user; AIShell-Gate can permit `git status` and `git diff` at confirmation level `none`, require action confirmation for `git push`, and deny `git clone` from an external URL entirely — and can do this differently for SSH sessions vs. TTY sessions, for specific UIDs or GIDs, and within a declared time window. AIShell-Gate also maintains a tamper-evident audit chain of every decision, which `sudoers` does not. The two mechanisms can coexist.

IT IS NOT A FULL INTRUSION PREVENTION SYSTEM

An IPS inspects running traffic or system calls in real time and can block actions mid-execution. AIShell-Gate operates at the command level, before execution begins. It does not observe what a command does while running or intercept system calls.

IT IS NOT A GUARANTEE AGAINST MALICIOUS BEHAVIOR

A sufficiently determined attacker with access to the system can attempt to bypass any software policy layer. AIShell-Gate is hardened — it validates its own binary, refuses setuid operation, enforces the evaluation boundary, and produces a linked audit chain — but it is not a security proof. It raises the cost and visibility of unsafe AI-generated actions and ensures that every attempt is recorded.

10 Threat Model

AIShell-Gate addresses a specific and bounded threat: the execution of AI-generated shell commands in environments where Unix infrastructure is deterministic and changes may be irreversible. The threat model is stated explicitly so that operators can reason about what AIShell-Gate covers and what complementary controls are required. The policy enforcement point model follows the access control framework described in NIST SP 800-162.[5]

ASSUMED ATTACKER AND ENVIRONMENT

AIShell-Gate assumes that the AI agent generating commands is probabilistic and may produce

dangerous, incorrect, or adversarially influenced output. It does not assume the AI is malicious in the traditional sense — it assumes the AI can be wrong, manipulated through prompt injection,[6] or directed toward a task whose full consequences it cannot evaluate. The threat is structural, not intentional.

THREAT CATEGORY	EXAMPLE	AISHELL-GATE RESPONSE
Dangerous command generation	`rm -rf /`, `dd if=/dev/sda | nc …`	DENY or CONFIRM_TYPED via catalog + arg rules
Shell metacharacter injection	`git status; rm -rf ~`	Rejected by `contains_shell_meta()` before tokenisation
Argument-level bypass	`find . -exec rm -rf {} \;`	Blocked by arg rules: `find -exec` denied
Interpreter bypass via env	`env python3 -c 'import os; os.system("bash")'`	Blocked by arg rules: `env python3` denied
Path traversal outside jail	Write to `/etc/cron.d/` via allowed command	Path rules deny writes to protected directories
Cron persistence without crontab	Direct write to `/var/spool/cron/`	Path rule denies `/var/spool/cron/` explicitly
Kernel memory access	Read/write via `/proc/PID/mem` or `/proc/kcore`	Path rules deny both entries explicitly
Self-Referential API Access (unaware)	`curl localhost:11434/api/generate` or `curl https://api.<provider>.com/…` during AI session	Port catalog covers local inference; planned hostname catalog covers remote and same-model. Loopback detection normalizes aliases. `CONFIRM_ACTION` minimum on match — operator informed and approves or denies; see §11
Cloud metadata access	`curl 169.254.169.254/latest/meta-data/`	Net rules deny all cloud IMDS endpoints
Network tool as backdoor	`nc -l 4444`	Arg rules deny `nc -l` / `--listen`
Audit trail falsification	`make -t` to forge build timestamps	Flag catalog: `make -t` is FLAG_DANGER
Timestamp forgery	`touch -t 202001010000 evidence.log`	Flag catalog: `touch -t` is FLAG_WARN with forensic reason

EXPLICIT SCOPE BOUNDARIES

In scope: AI-generated shell command evaluation, argument-level bypass attempts, path traversal within the execution environment, network target manipulation, AI self-referential API access, audit chain integrity.

Out of scope: Kernel exploits and privilege escalation below the process level, network-layer attacks, physical access, vulnerabilities in the AI model itself, and compromise of the operator's terminal or

confirmation channel. AIShell-Gate operates at command evaluation time; it does not monitor running processes or intercept system calls after execution begins.

> **THE CORE INSIGHT**
>
> *Classic reference monitors protect deterministic systems from other deterministic actors. AIShell-Gate protects a deterministic system from a probabilistic one. The threat is not that an attacker controls the AI — the threat is that the AI's probabilistic nature is itself a risk surface when connected to irreversible infrastructure. As Gilley argues in the context of safety-first system design, the architecture of safety must precede the architecture of capability — safety constraints retrofitted to existing capability are structurally weaker than those that shaped the capability from the beginning.*[10]

11 Model Sub-Delegation and Self-Referential API Access

This section introduces two related concepts that this paper proposes as distinct, named entries in the AI security threat taxonomy.

Model Sub-Delegation is the threat class when it occurs without awareness or explicit intent: an AI agent recruiting a second inference system — local or remote — to influence a policy decision that the original agent cannot influence directly, without the human operator knowing that a second model is involved in the decision. The concern is not the practice itself. A human operator who deliberately routes a command through a local model for a second opinion, adversarial testing, or ensemble reasoning is employing a sophisticated and potentially valuable strategy. The problem is when this occurs implicitly, inside a gated execution session, without the operator's knowledge — making the policy boundary's effective authority unclear.

Self-Referential API Access is the mechanism: the agent issues shell commands that contact inference endpoints — local or remote, and critically including the agent's own model provider — from within a gated execution session. The mechanism covers three increasingly subtle variants, in order of the sharpness of the self-reference:

- **Local secondary inference.** An agent issues `curl localhost:11434/api/generate` to consult a local Ollama instance, or any other local-inference runtime on a loopback port. A second model — a different system from the one running the agent — participates in the execution decision.
- **Remote secondary inference.** An agent issues `curl https://api.openai.com/v1/chat/completions` or a similar request to a different provider's API. The second model is remote, but still structurally distinct from the agent.
- **Same-model self-reference.** An agent powered by a given model issues a request to the *same* model through the *same* provider's public API — for example, an agent running on Claude issuing `curl https://api.anthropic.com/v1/messages`, potentially with the agent's own credentials. The second consulting system *is* the first system, consulted through its own public interface, which may be used to rephrase a denied command, propose a functionally equivalent alternative, or otherwise launder a policy decision that the session context would not allow directly.

The three variants share a common structure — an AI agent inside a gated session recruiting an inference system to influence a policy decision — and differ only in whether the recruited system is local, a different provider, or the same model consulted through its own API. The same-model case is the most subtle: the traffic looks like ordinary API use, the credentials may be legitimate, and the

recruited judgment is not traceably a different system. The threat is substantively identical across all three; the coverage required to detect them is not.

AIShell-Gate's position on this is consistent with its broader design philosophy: the gate does not prohibit, it makes explicit. Self-Referential API Access to an AI inference endpoint during an AI-sourced session triggers a confirmation requirement — the human operator is informed and must approve. If the operator intended the sub-delegation, they approve and it proceeds. If they did not, the gate catches something they would want to know about. The typed confirmation challenge, displayed only on the operator's terminal, ensures that the approval is genuinely human regardless of what the AI agent can observe. This distinction — between delegating generation, which is acceptable, and delegating judgment, which is not — is the ethical boundary that the confirmation gate enforces.[10] When an AI agent attempts to recruit a second model for a policy decision, it is attempting to delegate judgment to a system that cannot be held accountable for the consequences.

Model Sub-Delegation occurring through Self-Referential API Access — specifically the unaware or uncontrolled form — is a threat that emerged with the widespread deployment of autonomous and semi-autonomous AI agents with shell access. Local inference made the first variant (local secondary inference) broadly practical in 2023–2024. The remote and same-model variants are older in raw capability — any agent that can issue an HTTPS request can in principle reach any inference API — but became operationally significant only as agent-shell workflows became common, because without a shell-execution surface the self-reference vector has nowhere to land. The threat has no coverage in conventional security tooling, which is concerned with what commands an AI generates, not with whether the AI is consulting additional models during execution. It is related to but distinct from prompt injection[6] — prompt injection operates at the language model input layer; Self-Referential API Access operates at the execution layer, using shell commands rather than crafted text to recruit a second model. It is also related to but more specific than the OWASP LLM Top 10 category of Excessive Agency[11] — Excessive Agency addresses AI systems taking consequential actions beyond their intended scope; Model Sub-Delegation addresses the narrower case where an AI agent specifically recruits a second inference system to influence a policy decision governing the first agent's own execution session.

AIShell-Gate addresses this through a layered loopback and port-awareness system introduced in v1.01 and substantially expanded in v1.02.

THE PORT CATALOG

Version 1.02 replaces the original 25-entry AI port table with a 108-entry `g_port_catalog[]` covering five categories:

CATEGORY	ENTRIES	EXAMPLES
PORT_CAT_AI_INFERENCE	40	Ollama (11434), LM Studio (1234), llama.cpp (8080), Gradio (7860), Qdrant (6333), Chroma (8000), LocalAI, Jan.ai, Oobabooga, LiteLLM, AnythingLLM
PORT_CAT_WELL_KNOWN	32	SSH, SMTP, DNS, MySQL, PostgreSQL, Redis, MongoDB, Elasticsearch, Kafka
PORT_CAT_DANGEROUS	21	Telnet (23), rsh (514), Docker daemon unauthenticated (2375), Metasploit (4444), Memcached (11211), backdoor ports (31337, 12345)
PORT_CAT_CLOUD_META	3	AWS/GCP/Azure IMDS (169.254.169.254), AWS IPv6 IMDS (fd00:ec2::254),

		GCP alias
`PORT_CAT_DEBUG`	12	Node.js inspector (9229/9230), Tor control (9051), RDP (3389), Squid proxy (3128)

WARNING AND ESCALATION LOGIC

Network target detection operates in priority order for every evaluated command:

CONDITION	RESPONSE
`Cloud metadata host (any port)`	CONFIRM_TYPED — always
`PORT_RISK_DANGER port on any host`	CONFIRM_TYPED — regardless of target
`Loopback + AI inference port, source=ai`	CONFIRM_ACTION minimum — human must approve
`Loopback + PORT_RISK_DANGER port`	CONFIRM_TYPED
`Loopback + other known port`	Named service note, no automatic escalation
`Local alias (0.0.0.0, docker.internal)`	Advisory note

CURRENT COVERAGE AND ROADMAP: THE HOSTNAME CATALOG EXTENSION

The port catalog fully addresses the local secondary inference variant. It does not, by itself, address the remote and same-model variants, because those requests land on HTTPS port 443 — which is `PORT_CAT_WELL_KNOWN`, not `PORT_CAT_AI_INFERENCE`. An agent that issues `curl https://api.anthropic.com/v1/messages` during a gated session produces network target `api.anthropic.com:443`, which is not automatically escalated by the port-based logic alone.

The structural defense for the remote and same-model cases is a hostname and URL-pattern catalog that complements the port catalog. The same `g_port_catalog` infrastructure extends naturally to a parallel `g_hostname_catalog` with the same category set, recognizing major AI-provider API endpoints by hostname or URL pattern. A representative first-version hostname catalog would include `api.anthropic.com`, `api.openai.com`, `generativelanguage.googleapis.com`, `api.together.xyz`, `api.cohere.com`, `api.groq.com`, `api.deepinfra.com`, `api.mistral.ai`, `api.perplexity.ai`, `api.x.ai`, and similar provider endpoints, each categorized as `PORT_CAT_AI_INFERENCE` regardless of port.

With the hostname catalog in place, the escalation logic from the table above generalizes: the same *loopback + AI inference endpoint, source=ai* → `CONFIRM_ACTION` rule that fires for local Ollama fires equally for remote `api.openai.com`. The same source-tagging mechanism distinguishes agent-originated requests from operator-originated ones, allowing human use of AI APIs while gating agent self-reference. The same channel-isolated typed confirmation applies to the highest-risk case: an agent requesting the *same* model through the *same* provider can be automatically escalated to `CONFIRM_TYPED`, because that case is the sharpest form of Model Sub-Delegation and warrants the strongest friction.

This extension is a planned enhancement. The current 1.0 Beta implements the port catalog and the loopback-alias normalization; the hostname catalog is architecturally adjacent and expected in a subsequent release. The threat-model articulation in this section applies to all three variants; the current implementation coverage is complete for the local variant and partial for the remote and same-model variants pending the hostname catalog extension.

WHY THIS MATTERS

The confirmation challenge for a high-risk command is derived from the validated argv and displayed only on the human operator's terminal — it is never transmitted on the AI's communication channel. This means an AI agent cannot read the challenge and replay it. The human confirmation gate is structurally inaccessible to the model generating the command, regardless of what the model can see or say.

`PORT_RISK_DANGER` ports escalate to `CONFIRM_TYPED` on any target host, not only loopback. An AI agent cannot route around the loopback restriction by aliasing to `0.0.0.0` or `host.docker.internal` — both are explicitly covered by the loopback detection logic. Cloud metadata endpoint detection fires independently of port, so a request to `169.254.169.254` on any port is caught regardless of what service it claims to address.

> **THE GATE MAKES IT EXPLICIT, NOT IMPOSSIBLE**
>
> *Every other AI safety tool is concerned with what an AI says. AIShell-Gate is also concerned with what an AI calls. Model Sub-Delegation through Self-Referential API Access is not inherently wrong — a human operator who deliberately routes execution decisions through a local model is employing a legitimate advanced strategy. The problem is when it happens without the operator's awareness. The confirmation requirement does not prohibit the practice; it ensures the operator knows it is occurring and consciously permits it. That distinction — between an informed decision and an invisible one — is the boundary AIShell-Gate enforces.*

12 Source Code: Security in Practice

The following excerpts are drawn from the AIShell-Gate policy engine and executor source (v1.02 beta, policy 1.09 / exec 0.42.0). Each illustrates a specific, deliberate security decision embedded in the implementation. The pattern throughout: security properties are structural, not advisory. They are enforced in code, not described in documentation.

12.1 SHELL METACHARACTER REJECTION [AISHELL-GATE-POLICY.C]

Before tokenization, every input string passes through `contains_shell_meta()`. The function explicitly rejects every character class that could introduce shell injection — including the DEL byte (`0x7f`) added in v0.52 after fuzz testing[8] exposed the gap. Quoting characters are rejected entirely. This is a deliberate design choice: implementing a shell grammar subset to handle quotes would introduce ambiguity that becomes a bypass surface. Any command containing a quote character is denied outright.

```c
/* aishell-gate-policy.c — contains_shell_meta() */
static bool contains_shell_meta(const char *s) {
    for (const unsigned char *p = (const unsigned char *)s; *p; p++) {
        unsigned char c = *p;
        /* Reject all C0 controls except \t (0x09, treated as whitespace). */
        if (c < 0x20 && c != '\t') return true;
        /* DEL (0x7f) — not a C0 control but equally non-printable.       */
        /* No legitimate shell command uses 0x7f. Reject before tokenize. */
        if (c == 0x7f) return true;
        switch (c) {
            case '|': case ';': case '&':
            case '>': case '<': case '`':
```

```
            case '"': case '\'': return true;
        }
    }
    if (strstr(s, "$(") != NULL) return true;
    if (strstr(s, "${") != NULL) return true;
    if (strstr(s, "&&") != NULL) return true;
    if (strstr(s, "||") != NULL) return true;
    return false;
}
```

12.2 JAIL-ROOT ESCAPE PREVENTION [AISHELL-GATE-POLICY.C]

A subtle prefix-matching bug allowed a path like `/tmp/jailbreak/x` to satisfy a jail root of `/tmp/jail` — the first nine bytes match, so a bare `strncmp` passes. The fix confirms that the character immediately following the matched prefix is either `'/'` or `'\0'`, establishing true directory containment.

```
/* aishell-gate-policy.c — jail containment check                    */
/* v0.53 fix: strncmp prefix alone is insufficient.                  */
/* If jail_root is "/tmp/jail", the path "/tmp/jailbreak/x" matches  */
/* the first 9 bytes and incorrectly passes. The trailing character  */
/* must be '\0' or '/' to confirm containment under jail_root.       */
if (strncmp(canon, jail_root, jl) != 0 ||
    (canon[jl] != '\0' && canon[jl] != '/')) {
    argv_free(&out.argv);
    return deny_result("path is outside jail root");
}
```

12.3 JSON PARSE LOOP BOUNDS GUARD [AISHELL-GATE-POLICY.C]

All four rule-array parse loops previously relied on a sentinel token with `start == -1` at the end of the jsmn token array. When the array was exactly full, no sentinel existed and the loop read past the allocated buffer — a heap-buffer-overflow caught by AddressSanitizer[9] during a v0.53 audit pass.

```
/* aishell-gate-policy.c — parse loop bounds (all four rule-array parsers) */
/* `cur < g_tok_nt` guard: prevents heap overread when the token array  */
/* is exactly full and no sentinel token exists at end of the buffer.  */
while (cur < g_tok_nt && toks[cur].start != -1
       && toks[cur].start < endpos)
    cur++;
```

12.4 POLICY KEY ALLOWLIST [AISHELL-GATE-POLICY.C]

Unknown root keys in a policy file previously produced a silent no-op. A typo like `"cmd_denny"` was accepted without error and the intended rule never fired — a particularly dangerous failure mode because the operator believed they had a protection that did not exist. `validate_root_keys()` now rejects any key not on a 14-key allowlist, naming the offending key in the error. The allowlist includes `net_default_deny`, allowing policy files to configure the network enforcement model per project.

```
/* aishell-gate-policy.c — validate_root_keys() */
static const char * const known_keys[] = {
    "session",
    "cmd_allow",  "cmd_allow_replace",
    "cmd_deny",   "cmd_deny_replace",
    "arg_rules",  "arg_rules_replace",
```

```
    "path_rules", "path_rules_replace",
    "net_rules",  "net_rules_replace",
    "writable_dirs", "writable_dirs_replace",
    "net_default_deny",  /* stack-level bool — true=deny unknown net targets */
    NULL
};
/* Any key not on this list fails with a named error — not a silent no-op. */
if (!found) return false; /* unknown key — fail closed */
```

12.5 ATOMIC POLICY APPLY [AISHELL-GATE-POLICY.C]

Policy files are applied section-by-section. A failure midway through previously left the running policy in a half-written state. `policy_layer_snapshot()` takes a deep copy of the entire layer before any modification; any parse failure triggers `policy_layer_restore()`, returning the layer to its exact prior state. A failed config load has zero effect on the running policy.

```
/* aishell-gate-policy.c — policy_layer_snapshot/restore */
/* Take a full deep-copy before touching the layer.                    */
/* Any failure restores L to its original state — fail-closed.         */
PolicyLayerDyn snap;
if (!policy_layer_snapshot(L, &snap)) {
    set_err(err, "out of memory during policy snapshot");
    return false;
}
if (!parse_override_file(js, toks, nt, L, path, err)) {
    policy_layer_restore(L, &snap); /* zero effect on running policy */
    return false;
}
```

12.6 COMPILE-TIME SAFE PATH AND ENVIRONMENT ALLOWLIST [AISHELL-GATE-EXEC.C]

The executor never inherits the caller's PATH or environment. A compile-time directory list defines all searchable binary locations, and only an explicitly enumerated set of environment variables is propagated to executed commands. The approach is an allowlist, not a denylist: any variable not named is dropped, including all dynamic-linker and interpreter injection vectors. PATH itself is reconstructed from the compile-time list — it is never read from the environment.

```
/* aishell-gate-exec.c — SAFE_PATH and ENV_ALLOWLIST */
/* Fixed at compile time: a manipulated $PATH cannot redirect execution. */
static const char *const SAFE_PATH[] = {
    "/usr/local/bin", "/usr/bin", "/bin",
    "/usr/sbin", "/sbin", NULL
};
/* Allowlist: any variable absent is silently dropped, including        */
/* LD_PRELOAD, DYLD_INSERT_LIBRARIES, PYTHONPATH, GIT_EXEC_PATH.       */
static const char *const ENV_ALLOWLIST[] = {
    "HOME", "USER", "LOGNAME", "TERM", "COLORTERM",
    "LANG", "LC_ALL", "LC_CTYPE", "TZ", "TMPDIR", NULL
};
```

12.7 PRE-EXECUTION SECURITY SELF-CHECK [AISHELL-GATE-EXEC.C]

Before any policy evaluation begins, `check_execution_security()` validates the executor's own runtime posture. It refuses to continue if the process was elevated via setuid or setgid bits. It also rejects a binary that is writable by group or others. Both checks are written to the audit log before exit.

```c
/* aishell-gate-exec.c — check_execution_security() */
/* Check 1: refuse to run setuid or setgid. */
uid_t ruid = getuid(), euid = geteuid();
if (ruid != euid) {
    fprintf(stderr, "[gate-exec] SECURITY: refusing to run setuid\n");
    ok = false;
}
/* Check 2: binary not writable by group or others.               */
/* A writable binary can be replaced to bypass the policy engine. */
if (st.st_mode & (S_IWGRP | S_IWOTH)) {
    fprintf(stderr, "[gate-exec] SECURITY: binary is writable — fix: chmod go-w\n");
    ok = false;
}
```

12.8 RESPONSE STREAM CAP WITH SIGKILL [AISHELL-GATE-EXEC.C]

`slurp_timed()` reads the policy engine response under two independent constraints: a wall-clock timeout and a configurable byte cap. When either limit is reached, the child receives `SIGKILL` — not `SIGTERM` — because `SIGKILL` cannot be caught, blocked, or ignored. Buffer growth is also capped at `max_bytes + 4096` so `realloc()` cannot exhaust memory before the kill fires.

```c
/* aishell-gate-exec.c — slurp_timed() byte-cap enforcement */
if (total >= max_bytes) {
    fprintf(stderr, "[gate-exec] policy engine response exceeded %zu bytes — ",
            max_bytes);
    fprintf(stderr, "sending SIGKILL\n");
    /* SIGKILL cannot be caught, blocked, or ignored — unlike SIGTERM. */
    kill(kill_pid, SIGKILL);
    int ws = 0; waitpid(kill_pid, &ws, 0);
    free(buf); return NULL;
}
/* Buffer growth capped so realloc() cannot exhaust memory before kill. */
size_t new_cap = (cap < max_bytes) ? cap * 2 : max_bytes + 4096;
```

12.9 AUDIT JSON FIELD ESCAPING [AISHELL-GATE-EXEC.C]

User-controlled strings — the plan source field, policy version, layer names — are passed through `audit_json_str()` before being embedded in audit log entries. Without this, a malicious source field could inject arbitrary JSON into the audit chain, corrupting records or forging entries.

```c
/* aishell-gate-exec.c — audit_json_str() injection prevention          */
/* User-supplied input (source, layer, policy_version) could previously */
/* inject arbitrary JSON into audit entries. All external fields are    */
/* passed through audit_json_str() before use in snprintf format strings.*/
char esc_source[512], esc_layer[256], esc_version[256];
audit_json_str(esc_source,  sizeof esc_source,  plan->source);
audit_json_str(esc_layer,   sizeof esc_layer,   action->layer);
audit_json_str(esc_version, sizeof esc_version, policy_version);
snprintf(buf, sizeof buf,
    "{\"source\":\"%s\",\"layer\":\"%s\"}",
    esc_source, esc_layer);
```

12.10 INTERPRETER BYPASS VIA ENV [AISHELL-GATE-POLICY.C]

`env` is an allowed command. Without explicit arg rules, an AI agent could invoke `env python3`, `env bash`,

or any other interpreter via `env`, routing around a command-level deny for those interpreters entirely. The base arg rules table was expanded with eight explicit deny entries: `env bash`, `env sh`, `env dash`, `env zsh`, `env python`, `env python3`, `env perl`, and `env node`. The documented residual: `env VAR=x bash` with interleaved variable assignments is not caught by arg_glob matching and requires a path_rules or project-layer rule if that form is a concern in a given deployment.

12.11 FIND -EXEC AND FIND -DELETE AS COMMAND-LEVEL BYPASS [AISHELL-GATE-POLICY.C]

`find` is commonly allowed as a read-oriented filesystem search tool. Two flag combinations convert it into an execution primitive: `find -delete` achieves recursive deletion of matched files without invoking `rm`, routing around the `rm` command-level deny. `find -exec` and `find -execdir` chain arbitrary commands against every matched path, routing around command-level denies for the chained command entirely. Both are blocked by arg rules in the base policy; the distinction from the `rm` deny is that these bypass vectors operate on the `find` command, which is itself allowed.

12.12 /PROC KERNEL EXPOSURE AND /VAR/SPOOL/CRON PERSISTENCE PATH [AISHELL-GATE-POLICY.C]

`/proc/PID/mem` allows read and write access to any running process's virtual address space, bypassing all filesystem ACLs — it is the primary mechanism for credential extraction from running processes. `/proc/kcore` exposes the full kernel address space as an ELF core image. Neither was covered by the original path rule set. Both are now explicitly denied. Separately, `/var/spool/cron/` allows establishing cron persistence by writing directly to the cron spool without invoking `crontab`, routing around the `crontab` command-level deny. An explicit path deny for `/var/spool/cron/` closes this vector independently of the command rules.

12.13 AUDIT CHAIN-STATE UPDATE OUTSIDE THE FLOCK CRITICAL SECTION [AISHELL-GATE-POLICY.C]

In `audit_append_jsonl()`, the chain state variables `g_prev_hash` and `g_audit_seq` were updated after `flock(LOCK_UN)` and `close(fd)` — outside the exclusive lock. Under concurrent evaluation, two policy processes could write audit entries with the same sequence number and matching previous-hash value, producing a chain that verifies correctly per-entry but contains duplicate sequence numbers at the race site. The fix updates both state variables while the exclusive lock is still held, immediately after the write loop completes and before `flock(LOCK_UN)` — matching the implementation in the executor binary.

12.14 MAKE -T: AUDIT TRAIL FALSIFICATION [AISHELL-GATE-POLICY.C]

`make -t` (touch mode) updates file timestamps without executing any build commands. In an audited environment this creates a record that files were built at a particular time when no build occurred, allowing an actor to backdate build artifacts or conceal that a build step was skipped. This is a distinct threat category from data destruction or privilege escalation: the risk is evidence manipulation. `make -t` is catalogued as `FLAG_DANGER` with the reason `"touch mode updates timestamps without executing commands — creates misleading audit trail about what was built when."`

13 Design Principles

AIShell-Gate is intentionally simple: fail closed, deterministic rule evaluation, human-readable reasoning, minimal external dependencies, transparent C implementation, no hidden AI inference. Both source files embed JSMN, a minimal MIT-licensed JSON tokenizer, verbatim rather than linking an external library. This eliminates a supply-chain dependency: a build-system or package-manager attack

cannot substitute a malicious parser.

A governing design principle throughout is *Epistemic Honesty*: the system never implies it evaluated something and found it safe when it did not. An unknown flag produces a `FLAG_UNKNOWN` output explicitly labelled "risk unassessed, not evaluated as safe." Unknown root keys in a policy file are rejected with a named error rather than silently ignored. A failed policy load produces an explicit error rather than a silent fallback. Epistemic Honesty is closely related to the open design principle of Saltzer and Schroeder[4] — security mechanisms should not depend on obscurity, and their behaviour should be fully visible and explainable to the operator. It also reflects the observation that trust emerges from inspectability: a system that explains its reasoning is safer than one that merely performs well, because the explanation supports the oversight that catches what performance alone cannot guarantee.[10]

A related principle governs the design of enforcement rules: a control that can be bypassed by a sufficiently unusual input is not a control — it is a preference, and preferences do not hold under adversarial conditions or at scale.[10] This is the argument behind Zero-Effect Fail-Closed design and the layered arg rules that close bypass vectors independently of the command-level deny rules they complement.

The goal is not intelligence. The goal is clarity. In environments where AI generates actions, clarity is more important than cleverness. An operator should be able to read a policy file, understand exactly what it permits and denies, and trust that the engine applies it literally — not probabilistically.

The executor-separation principle follows from this directly: the policy engine validates, the executor acts. The boundary between them is the JSON decision record, which is structured, logged, and auditable. Neither component reaches across that boundary.

> **DESIGN CONSTRAINTS**
>
> *AIShell-Gate is intentionally constrained by several principles drawn from practical Unix usage. The system must remain deterministic, simple enough to audit, compatible with shell pipelines, and capable of falling back to explicit human confirmation for risky actions. These constraints intentionally exclude more complex approaches such as AI-based risk evaluation or heuristic command interpretation. The goal is not to build an intelligent system, but a predictable boundary that integrates cleanly into existing Unix workflows. It is this deliberate simplicity — not sophistication — that makes the boundary trustworthy.*

14 Who Needs This

AIShell-Gate is intended for environments where AI systems generate or propose shell commands that may affect real infrastructure, data, or operational workflows. It is most useful in situations where organizations want to benefit from AI-assisted operations without allowing probabilistic systems to directly control deterministic infrastructure.

AI-ASSISTED DEVOPS TEAMS

Modern development teams increasingly use coding assistants that can suggest shell commands or propose multi-step workflows — dependency updates, build system repairs, log inspection, test execution, repository maintenance. Without a policy layer, the typical workflow is: AI suggests command → developer copy/pastes → command executes. This introduces risk: incorrect flags, incorrect

paths, destructive commands suggested during debugging, accidental system modification outside the project directory.

With AIShell-Gate the workflow becomes:

```
AI proposes command → policy evaluates → executor runs validated argv
```

COMMAND	RESULT
`git status`	ALLOW — no confirmation
`npm test`	ALLOW — no confirmation
`rm -rf node_modules`	ALLOW — confirmation required
`rm -rf /`	DENY

ORGANIZATIONS RUNNING AUTONOMOUS OR SEMI-AUTONOMOUS AGENTS

Some organizations are beginning to deploy autonomous agents that perform operational tasks: infrastructure maintenance, dependency management, repository triage, automated diagnostics, CI/CD repair. These agents may generate shell commands dynamically based on system state. AIShell-Gate provides a policy enforcement boundary that prevents an agent from executing actions outside defined operational limits.

```
AI Agent | JSON plan ▼ aishell-gate-exec | ▼ aishell-gate-policy (policy decision) | ▼
execve(validated argv)
```

This model ensures every command is validated, destructive actions require confirmation, forbidden operations never execute, and all decisions are recorded in the audit log.

SECURITY-CONSCIOUS INFRASTRUCTURE TEAMS

Security teams are increasingly concerned about the introduction of AI into operational environments. The core concern is simple: AI systems produce probabilistic output, while infrastructure requires deterministic control. AIShell-Gate introduces a deterministic policy layer that enforces allowed command sets, allowed argument patterns, path restrictions, network target restrictions, session policy constraints, and confirmation escalation based on risk.

In practice: `dd` or `mkfs` commands automatically receive risk scores of 95–98 and require typed confirmation. Recursive deletion of system paths is denied outright. Writes to protected directories require explicit human approval. Every decision is recorded in a tamper-evident audit chain.

CONTROLLED REMOTE AI EXECUTION ENVIRONMENTS

Some organizations want to allow external AI systems to interact with internal infrastructure through a controlled interface. AIShell-Gate enables a model where the AI never receives direct shell access. Instead the AI submits structured execution requests; the executor evaluates each command through the policy engine before execution; and only the validated argv returned by the policy engine is ever

passed to `execve()`. This prevents shell injection, environment inheritance, and arbitrary command execution.

ENGINEERING TEAMS BUILDING AI-DRIVEN TOOLCHAINS

AIShell-Gate is also useful as a building block for larger systems: AI-powered developer environments, automated incident response tools, infrastructure maintenance agents, experimental AI operations platforms. Because the policy engine and executor are separate processes, the system can be embedded into larger architectures while maintaining the policy boundary. AIShell-Gate acts as the decision engine; surrounding systems provide orchestration, monitoring, sandboxing, network controls, and authentication.

HUMAN OPERATORS — SAFETY NET, TEACHING TOOL, AND DISCIPLINED WORKFLOW

AIShell-Gate is not exclusively an AI safety tool. The same policy engine, the same confirmation gates, and the same audit chain apply equally when a human operator types commands interactively at a terminal. This is not a secondary use case — it is a distinct user population served simultaneously by the same binary with no additional configuration.

For human operators, AIShell-Gate serves three functions at once:

Safety net. Dangerous commands typed by a human are caught before execution by exactly the same rules that catch dangerous AI-generated commands. A tired operator at 2am during incident response who types `rm -rf /var/log/*` receives the same denial as an AI agent that proposes it. The gate does not discriminate by source.

Teaching tool. Every flag assessment in the 2,088-entry flag catalog carries a documented reason explaining *why* the flag is categorized as it is — not merely what the decision is. A junior operator who types `sed -i file.txt` is told "in-place edit; writes back to source file." Someone who uses `touch -t` learns "timestamp forgery can backdate files to evade time-based audit trails." `dmesg -c` produces "clears the kernel ring buffer after reading — destroying evidence impedes forensic investigation." `make -t` is flagged as "touch mode updates timestamps without executing commands — creates misleading audit trail about what was built when."

This is not documentation that gets read separately. It is instruction delivered in context, at the moment of the command, naming the specific flag that triggered the concern and the specific risk it represents. The gate teaches the reasoning behind Unix security practice, not just the outcome. Equally, `FLAG_UNKNOWN` is explicitly labelled "risk unassessed, not evaluated as safe" — teaching the practitioner that unknown is not the same as safe. That epistemic distinction is a professional maturity lesson most people take years to learn.

Disciplined workflow. Experienced operators gain confirmation gates and a tamper-evident audit trail on their own commands — not just on AI-generated ones. Every interactive command is assessed, every decision is logged, and high-risk operations require typed confirmation derived from the exact command being approved. This provides compliance-grade accountability for human shell activity without requiring any change to existing workflow.

The policy governing all three functions — AI agent, autonomous pipeline, human operator — is the same file, evaluated by the same engine. An organization that sets a base policy for AI agent safety automatically extends that policy to human operator sessions. There is no separate human-mode

configuration.

> **IN SHORT**
> *AIShell-Gate serves two distinct user populations from a single unified policy engine: AI agents submitting structured execution plans, and human operators working interactively at a terminal. For AI agents it provides a deterministic policy boundary between probabilistic output and irreversible infrastructure. For human operators it provides a safety net, a teaching instrument, and a disciplined workflow with full audit accountability. Both populations are served by the same gate, the same policy, and the same audit chain — simultaneously, without additional configuration.*

15 Current Implementation Features

`aishell-gate-policy 1.09` · `aishell-gate-exec 0.42.0` · `aishell-gate-mcp 1.0`

> **Editions** AIShell-Gate ships in two editions. The **Standard edition** includes all policy evaluation, all built-in presets, all confirmation levels, interactive mode, audit logging (JSON Lines), jail-root enforcement, session policy, custom policy file layers, and `--dump-standard-template`. The **Enterprise edition** adds: `--dump-policy` (full resolved effective policy stack as JSON, including catalog-derived internals — useful for security review and policy debugging); HMAC-SHA256 keyed audit chain on the executor log (`--audit-key`); `--audit-verify` for chain integrity verification; and cryptographic session ID binding in the audit chain. Invoking an Enterprise-only flag on a Standard binary produces a clear "not available in standard edition" message, not a silent failure. Run `--version` on either binary to identify the edition.

Structured JSON Decision Channel. The policy engine produces machine-readable JSON describing session metadata and per-action decisions. The executor consumes JSON only and does not parse human-readable text.

Deterministic Execution Pipeline. The executor spawns the policy engine directly and controls the full evaluation → decision → execution pipeline. No external plan input bypasses this flow.

Absolute Path Resolution. Executable binaries are resolved against a controlled `SAFE_PATH` list and executed via `execve()` using absolute paths. `PATH` is never inherited from the caller's environment.

Hardened Execution Environment. The executor constructs an allowlisted environment from scratch. Any variable absent from the allowlist — including `LD_PRELOAD`, `DYLD_INSERT_LIBRARIES`, `PYTHONPATH`, and `GIT_EXEC_PATH` — is silently dropped.

Evaluation Timeout and Response Size Cap. Policy engine evaluation is time-bounded. The executor enforces a configurable byte cap on the policy engine response stream; when exceeded the child is `SIGKILL`'d before the deadline fires, preventing OOM exhaustion.

Typed Confirmation Challenge (Command-Coded Confirmation). Destructive or high-risk actions require a confirmation challenge derived from the validated command — the response is computed from the specific argv being approved, not a generic acknowledgement. The challenge is displayed only on the human operator's terminal and is never exposed to the AI's communication channel.

Zero-Effect Fail-Closed Design. Any parse failure, malformed JSON, policy ambiguity, or execution resolution failure results in denial. A failed policy config load has zero effect on the running policy state due to atomic snapshot/restore semantics — not only does the system deny, it leaves active policy

entirely unchanged. This is a stronger guarantee than standard fail-closed design, which specifies denial on failure but not the integrity of running state.

Tamper-Evident Audit Chain. Each audit entry is SHA-256 hashed and linked to the previous entry, forming a verifiable append-only chain. HMAC-SHA256 mode (`--audit-key`) restricts verification to key-holders. Concurrent writes are protected by advisory file locking.

Pre-execution Security Self-Check. The executor validates its own runtime state before proceeding: refuses setuid/setgid, detects writable-binary conditions, and logs violations to the audit trail.

Embedded JSON Tokenizer. Both components embed JSMN verbatim, eliminating a supply-chain dependency on an external parsing library.

Model Sub-Delegation Detection via Self-Referential API Access Gating. A 108-entry port catalog (40 AI inference endpoints, 21 inherently dangerous ports, 32 well-known services, 3 cloud metadata endpoints, 12 debug ports) identifies network targets in evaluated commands. Self-Referential API Access — loopback connections to AI inference ports — automatically escalates to `CONFIRM_ACTION` minimum when `source=ai`, preventing Model Sub-Delegation through this vector without human approval. Dangerous ports escalate to `CONFIRM_TYPED` on any target host. Cloud metadata endpoints are denied regardless of port. See §11.

Argument-Level Bypass Coverage. The base arg rule table covers 40 flag-level attack vectors, including: interpreter bypass via `env`, recursive deletion via `find -delete`, arbitrary execution via `find -exec`, persistent tunnel creation via `ssh -L/-R/-D/-w`, raw device read via `dd if=/dev/...`, and backdoor listener via `nc -l`.

Unified Human Operator Mode — Safety Net, Teaching Tool, and Disciplined Workflow. Both binaries operate interactively when stdin is a terminal. `aishell-gate-policy` interactive mode is a purely educational surface: it assesses commands, explains every flag with its documented reasoning, and emits a gateway echo line — it never executes. `aishell-gate-exec` interactive mode is the execution surface: it enforces confirmation gates and runs commands with a full audit trail. The same policy file governs both AI agent sessions and human operator sessions. The 2,088-entry flag catalog — where every `FLAG_WARN` and `FLAG_DANGER` entry carries a specific documented reason rather than a generic label — functions as an active teaching instrument: a junior operator who types an unfamiliar flag receives an explanation of precisely why that flag raises the risk profile, delivered at the point of use. `FLAG_UNKNOWN` is explicitly labelled "risk unassessed, not evaluated as safe," teaching practitioners that absence of a denial is not an implicit safety determination.

Dry-Run JSON (`--dry-run-json`). The executor produces a machine-readable JSON document on stdout describing every action that would execute — including the resolved binary path, validated argv array, confirm level, risk score, layer, and reason — without executing any of them. Designed for AI agents that inspect a plan before committing to live execution. Policy evaluation, confirmation checking, and the audit chain all run normally; only the OS-level execve() call is suppressed. The output carries the `aishell-gate-dry-run 1.0` protocol block so consumers can detect schema version.

Policy Test Mode (`--test-plan`). The policy engine accepts a JSON file of test cases and evaluates each against the active policy, reporting PASS or FAIL per case and exiting non-zero on any failure. Each test case specifies a command, an expected decision (allow or deny), an optional preset, and an optional label. Suitable for committing alongside policy files and running as a CI gate — a policy change that breaks an expected allow or deny is caught before deployment. Exit 0 means all tests passed; exit 1

means failures; exit 2 means file or parse error.

Network Default-Deny (`net_default_deny`). Network enforcement is decoupled from command enforcement. In `ops_safe`, `read_only`, and `dev_sandbox` presets, detected network targets must have explicit allow rules or the command is denied. CI presets set `net_default_deny` to false, allowing unrestricted network access for build pipelines. Configurable per-project via override files.

Wire Protocol Versioning. All four JSON interfaces carry a named protocol block: `aishell-gate-exec-input 1.0` (agent → exec), `aishell-gate-policy-response 1.0` (policy engine output), `aishell-gate-dry-run 1.0` (dry-run inspection output), and `aishell-gate-mcp 1.0` (MCP tool responses). Major version bumps signal breaking changes; minor bumps are additive only. The executor validates incoming envelopes and rejects unknown protocol names or unsupported major versions with a clear error.

Expanded Flag Catalog (2,088 entries). Additions cover ansible and ansible-playbook (54 entries including privilege escalation flags `--become` / `--become-user`), az/Azure CLI (33 entries with `--yes` as FLAG_DANGER), extended kubectl (34 additional entries including `drain`, `--force`, `run`, `proxy`), extended terraform (18 entries including `-auto-approve` and `-migrate-state`), extended helm (24 entries), insmod/rmmod/kexec (kernel module and boot chain operations, all FLAG_DANGER), fdisk/parted/sfdisk/sgdisk (disk partitioning), debugfs (VFS-bypass filesystem access), socat (bidirectional relay with full EXEC/SYSTEM/TCP-LISTEN coverage), nft/nftables (firewall rule management), the full LVM suite (lvcreate through pvremove), and extended aws/gcloud CLI flags.

16 Audit and Accountability

Every evaluation can be written to a tamper-evident JSON Lines audit log (`--audit-log`). Each record carries a monotonic sequence number, session identifier, full decision context, and a SHA-256 hash linking it to the preceding record. A gap in sequence numbers or a hash mismatch identifies deleted or altered entries.

For environments requiring authenticated audit trails, `--audit-key` upgrades the chain to HMAC-SHA256. Only a holder of the 64-byte key can forge valid chain hashes. The chain can be verified at any time with `--audit-verify` without interrupting production operation.

This provides a lightweight, dependency-free audit record suitable for compliance review, incident reconstruction, and policy tuning.

17 MCP Integration — aishell-gate-mcp

`aishell-gate-mcp` is an MCP (Model Context Protocol) server that exposes AIShell-Gate to AI coding environments — Claude Code, Cursor, and any client implementing the MCP standard — over a stdio transport. It requires no additional infrastructure: a single entry in the project's `.mcp.json` launches the server as a subprocess, and the AI coding environment discovers the tools automatically.

> **THE SCOPE OF THIS TOOL**
>
> *Every other MCP tool gives an AI access to one capability — a calendar, a database, a code search index. AIShell-Gate gives it access to the operating system, which is the thing that contains everything else. The MCP interface is what makes that access distributable to every AI coding environment in the ecosystem. The policy engine is what makes it safe.*

TWO TOOLS

`evaluate_plan` submits a goal and list of commands to the policy engine via `--dry-run-json`. It returns a structured assessment: per-action decision, confirm level, resolved binary path, risk score, layer, and reason. Nothing executes. The tool is designed as a pre-flight check — the agent can inspect the full plan assessment before committing to live execution, identify denied actions, and surface confirmation requirements to the operator.

`execute_plan` submits a goal and list of commands for live execution. It runs an internal pre-flight evaluation first. If any action requires confirm level `action` or `typed`, execution is blocked and a structured report is returned identifying the specific actions and the reason confirmation is required. The operator can then lower the confirm level in their policy file, approve the actions manually, or restructure the plan. If the pre-flight passes with only `none` or `plan` level actions, live execution proceeds via `aishell-gate-exec`.

Three additional tools expose policy-engine capabilities directly without execution involvement: `evaluate_command` submits a single command string for policy assessment and returns the full decision record — confirm level, risk score, matched rule, flag analysis, and reason — without constructing a plan envelope; `get_policy_template` returns the standard policy template as a JSON object, providing a starting point for custom policy file authoring; and `verify_policy` validates a policy file against the schema and reports any structural errors before deployment. These three tools are read-only and invoke only the policy engine, never the executor.

CONFIRMATION MODEL

The MCP interface enforces a block-and-report model for confirmation in v1.0. Actions at `action` level return a structured response listing every affected action and the reason confirmation is required; the operator is expected to review these and adjust policy or proceed manually. Actions at `typed` level are blocked unconditionally — typed confirmation requires a human at a terminal, which the MCP interface cannot provide. This is intentional: `typed` confirmation is designed to be structurally inaccessible to any automated channel. An operator who wants a class of commands to run through the MCP interface without blocking can lower the confirm level in their policy file for trusted workflows.

CONFIGURATION

An `aishell-mcp.json` file in the project directory configures the server. All fields are optional with sensible defaults:

```
// aishell-mcp.json — all fields optional
{
  "exec_binary":    "aishell-gate-exec",    // resolved via PATH if not absolute
  "policy_binary":  "aishell-gate-policy",
  "preset":         "ops_safe",
  "jail_root":      null,                   // confine writes to this directory tree
  "policy_base":    null,
  "policy_project": null,
  "policy_user":    null,
  "source":         "ai",
  "audit_log":      null,
  "extra_flags":    []
}
```

Claude Code / Cursor integration is a single entry in `.mcp.json`:

```
{
  "mcpServers": {
    "aishell-gate": {
      "command": "python3",
      "args": ["/path/to/aishell-gate-mcp.py"]
    }
  }
}
```

PROTOCOL VERSIONING

Every MCP tool response carries a protocol block: `{"name": "aishell-gate-mcp", "version": "1.0"}`. The server validates protocol blocks in exec responses and rejects mismatched names or unsupported major versions with a clear error. The full wire protocol inventory for v1.0:

PROTOCOL NAME	VERSION	DIRECTION
`aishell-gate-exec-input`	1.0	MCP server / agent → aishell-gate-exec (stdin)
`aishell-gate-policy-response`	1.0	aishell-gate-policy → aishell-gate-exec (stdout)
`aishell-gate-dry-run`	1.0	aishell-gate-exec → caller (stdout, --dry-run-json)
`aishell-gate-mcp`	1.0	aishell-gate-mcp → MCP client (tool responses)

Major version bumps signal breaking changes. Minor bumps are additive only; receivers must tolerate unknown minor versions without error. This versioning contract is in force as of v1.02 and will govern all future protocol evolution.

BROKER COMPATIBILITY (V2 SEAM)

In v1.0, the MCP server invokes `aishell-gate-exec` directly as a subprocess. A v2 broker daemon — planned as a third program — will accept multiple concurrent AI agents and human operators, with exec and the policy engine as a shared back end. The MCP server's broker seam is a single function: `_invoke_exec()`. In v2, replacing the subprocess call with a socket connection to the broker is a one-function change; all tool logic, protocol handling, and config loading are unchanged.

18 Status

AIShell-Gate 1.02 is a beta release. The architecture is stable and has been in iterative development and hardening since the initial prototype. All five build variants — standard policy, enterprise policy, standard exec, enterprise exec, and evaluation exec — compile clean against C11 with `-Wall -Wextra` and zero errors. The full test suite passes: 55 policy unit tests, 87 executor unit tests, 35 end-to-end tests, 34 catalog tests, and 12 standard-edition smoke tests. The policy model, execution gateway, audit chain, loopback detection system, MCP server, and all wire protocol interfaces are functional. Access is currently by arrangement while final pre-release validation is completed. Feedback from practitioners introducing AI into Unix workflows is welcomed and actively shapes the roadmap.

v1.02 additions since v1.01: aishell-gate-mcp MCP server; flag catalog expanded to 2,088 entries;

`net_default_deny` network enforcement model; `--dry-run-json` machine-readable plan inspection; `--test-plan` policy test mode; wire protocol versioning across all four JSON interfaces; Standard/Enterprise edition split compiled and verified.

Security inquiries and deployment questions: www.aishellgate.com · www.aishell.org · info@aishell.org · sgilley@aishellgate.com

> **CLOSING THOUGHT**
>
> *Trust in AI systems cannot be achieved by attempting to make probabilistic models perfectly reliable. Trust emerges from architecture: deterministic boundaries that govern how AI-generated actions interact with real systems. AIShell-Gate represents one such boundary — a policy layer that converts probabilistic suggestions into controlled, auditable system operations. As AI systems increasingly interact with infrastructure, databases, networks, and operational tooling, similar execution boundaries will likely become a necessary architectural component. AIShell-Gate demonstrates how such a boundary can be implemented for Unix command execution: simple, deterministic, and auditable. The MCP interface makes the point concrete. Every other MCP tool gives an AI agent access to one capability. AIShell-Gate gives it access to the operating system — the thing that contains everything else. The policy engine is what makes that access safe. Power without policy is risk. AIShell-Gate exists to make the boundary explicit.*

19 Terminology Glossary

The following terms are introduced or formally defined in this paper. Several fill gaps in existing security vocabulary where the concepts are genuinely novel or where existing terms are insufficiently precise for the AI execution context.

Model Sub-Delegation THREAT CLASS

The act of an AI agent recruiting a second inference system — local or remote — to influence a policy decision from within a gated execution session, without the human operator's knowledge or explicit approval. Model Sub-Delegation is not inherently prohibited; the concern is specifically the unaware form, where a second model participates in an execution decision invisibly. The mechanism by which this most commonly occurs in local AI deployments is Self-Referential API Access.

Introduced §11 · Related: Self-Referential API Access

Self-Referential API Access MECHANISM

The specific vector by which Model Sub-Delegation is attempted: an AI agent issues shell commands that contact inference endpoints from within a gated execution session. Three variants in order of the sharpness of the self-reference: (i) local secondary inference — e.g., `curl localhost:11434/api/generate` to a local Ollama; (ii) remote secondary inference — a different provider's API reached over HTTPS; (iii) same-model self-reference — the agent consulting its *own* model through the same provider's public API, potentially with its own credentials, laundering a policy decision through an apparently ordinary API call. AIShell-Gate detects the local variant through the 108-entry port catalog and loopback-alias normalization; the remote and same-model variants are addressed by a planned hostname catalog extension. Detection in all variants requires explicit human confirmation before the command executes, converting an invisible action into a conscious operator decision.

Introduced §11 · Related: Model Sub-Delegation, Port Catalog

Command-Coded Confirmation SECURITY MECHANISM

A confirmation challenge whose response string is computed from the validated argument vector of the specific command requiring approval. Because the challenge is derived from the argv, it cannot be pre-computed, replayed for a different command, or satisfied by a generic approval. The challenge is displayed only on the human operator's terminal and is never transmitted on the AI agent's communication channel, making it structurally inaccessible to the model generating the command. This is the `CONFIRM_TYPED` confirmation level in AIShell-Gate.

See §3, §15 · Implementation: `CONFIRM_TYPED`

Zero-Effect Fail-Closed DESIGN PROPERTY

A stronger form of the standard fail-closed design principle. In standard fail-closed design, a failure results in denial. In Zero-Effect Fail-Closed design, a failure results in denial *and* leaves the running system state entirely unchanged — no partial writes, no degraded policy, no intermediate state. In AIShell-Gate, a failed policy configuration load triggers atomic restoration of the prior policy state via snapshot/restore semantics, so a bad config file has zero effect on the currently active policy.

See §12.5 · Implementation: `policy_layer_snapshot/restore`

Declared Source Identity SESSION ATTRIBUTE

A session source identity set by the deployer or operator via a command-line flag (`--source ai` or `--source human`). Declared Source Identity is authoritative — it reflects a deliberate deployment decision rather than a claim made by the agent itself. Policy surfaces that activate based on source identity, including AI inference port escalation and audit attribution, treat the declared value as ground truth for the session.

See §6, §11 · Implementation: `--source` flag · Related: Injected Source Identity

---

Injected Source Identity SESSION ATTRIBUTE

A source identity claim supplied by the AI agent itself within the JSON plan (`"source": "ai"` or `"source": "human"`). Injected Source Identity is advisory and potentially adversarial — an agent that identifies itself as `human` in its plan is making an unverifiable claim. Deployments with strong source identity requirements should use Declared Source Identity via CLI flag rather than relying on agent-supplied values. The distinction between declared and injected source is a security property that policy authors should reason about explicitly.

See §6 · Implementation: JSON plan `source` field · Related: Declared Source Identity

---

Epistemic Honesty DESIGN PRINCIPLE

The design principle that a system must never represent an unevaluated state as a safe one. The term applies the philosophical concept of epistemic honesty — truthfulness about the limits of one's knowledge — to system design. The novel contribution is the specific design obligation it creates: where conventional security tools treat the absence of a denial as an implicit pass, AIShell-Gate treats it as an explicit gap. An unknown flag produces a `FLAG_UNKNOWN` output labelled "risk unassessed, not evaluated as safe." Unknown root keys in a policy file are rejected with a named error. A failed policy load produces an explicit error rather than a silent fallback. The philosophical source concept is established; its formalisation as a named system design principle in the AI execution security context is introduced here.

See §13 (Design Principles) · Related: Zero-Effect Fail-Closed, Default Deny · Philosophical lineage: epistemology, open design (Saltzer & Schroeder [4])

Execution Posture CONCEPT · SEE ALSO: POLICY PRESET

The complete risk and permission profile governing an AIShell-Gate session, encompassing the allowed command set, confirmation floors, path constraints, network rules, and source identity handling. An Execution Posture is the expression of an organisation's intent for a specific workflow type: what actions are permitted, what requires human

review, and what is denied regardless of context. AIShell-Gate implements Execution Postures as named Policy Presets (`ops_safe`, `dev_sandbox`, `ci_build`, `ci_deploy`, `ci_admin`, `read_only`, `danger_zone`). "Execution Posture" is the conceptual term; "Policy Preset" is the implementation term.

See §5, §6, §15 · Implementation: `--policy-preset` flag · Related: Policy Preset

---

Net Default-Deny POLICY PROPERTY

A network enforcement posture in which any command with a detected network target — a URL, `host:port`, or parseable address in its arguments — must have an explicit allow entry in the active `net_rules` policy for every target. Targets with no matching allow rule are denied rather than warned, mirroring the command policy model. Net Default-Deny is governed by the `net_default_deny` flag on the policy stack, which is decoupled from the command default-deny gate to allow CI presets to enable unrestricted network access for build pipelines without relaxing command policy. The built-in cloud metadata denial rules are unconditional and operate independently of `net_default_deny`.

Introduced §5 · Implementation: `net_default_deny` field · Related: Default Deny, Execution Posture

---

Broker Seam ARCHITECTURE PROPERTY

A deliberately isolated integration point in `aishell-gate-mcp` — a single function, `_invoke_exec()` — that handles all communication between the MCP server and the execution backend. In v1.0 this function invokes `aishell-gate-exec` as a local subprocess. In v2.0 it will be replaced with a connection to the broker daemon, which manages concurrent AI agent and human operator sessions over a shared execution back end. The broker seam design principle ensures that all protocol handling, tool logic, and configuration loading above the seam are unchanged when the underlying transport changes. The pattern is analogous to the policy/mechanism separation that governs the two-binary architecture.

See §17 · Implementation: aishell-gate-mcp · Related: Policy/Mechanism Separation

20 References

[1] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. "*Concrete Problems in AI Safety.*" arXiv:1606.06565, 2016. arxiv.org/abs/1606.06565 — Frames the structural gap between AI optimisation targets and human intent in operational environments; directly motivating the probabilistic/deterministic mismatch addressed by AIShell-Gate.

[2] Russell, S. *Human Compatible: Artificial Intelligence and the Problem of Control.* Viking, 2019. ISBN 978-0525558613. — Provides the broader context for why architectural boundaries, rather than model reliability alone, are the appropriate mechanism for AI safety in operational settings.

[3] Anderson, J. P. "*Computer Security Technology Planning Study.*" Technical Report ESD-TR-73-51, Air Force Electronic Systems Division, 1972. — Original formulation of the reference monitor concept: a trustworthy component that interposes between subjects and objects, enforcing complete mediation. AIShell-Gate adapts this model to the AI agent / Unix execution boundary.

[4] Saltzer, J. H. and Schroeder, M. D. "*The Protection of Information in Computer Systems.*" Proceedings of the IEEE, 63(9):1278–1308, 1975. — Defines the eight design principles for secure systems, including separation of policy and mechanism (directly reflected in the two-binary architecture) and open design (the foundation of Epistemic Honesty as applied here).

[5] Hu, V. C., et al. "*Guide to Attribute Based Access Control (ABAC) Definition and Considerations.*" NIST Special Publication 800-162, National Institute of Standards and Technology, 2014. doi.org/10.6028/NIST.SP.800-162 — Defines the policy enforcement point model and policy decision point separation that frames the AIShell-Gate threat model and architecture.

[6] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. "*Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections.*" arXiv:2302.12173, 2023.

arxiv.org/abs/2302.12173 — Characterises prompt injection as an attack on the language model input layer. Self-Referential API Access is a related but distinct vector: it operates at the execution layer, using shell commands to recruit a second model rather than crafted text to manipulate a single model's output.

[7] NIST. "*Artificial Intelligence Risk Management Framework (AI RMF 1.0).*" NIST AI 100-1, National Institute of Standards and Technology, 2023. doi.org/10.6028/NIST.AI.100-1 — Establishes the governance context for AI risk management in operational deployments; the Execution Posture concept in AIShell-Gate maps directly to the AI RMF's organisational profile and risk tier constructs.

[8] Miller, B. P., Fredriksen, L., and So, B. "*An Empirical Study of the Reliability of UNIX Utilities.*" Communications of the ACM, 33(12):32–44, 1990. — Foundational fuzz testing methodology applied to Unix command-line tools; the fuzz testing pass that discovered the DEL byte (0x7f) gap in AIShell-Gate follows the input-space exploration approach described here.

[9] Serebryany, K., Bruening, D., Potapenko, A., and Vyukov, D. "*AddressSanitizer: A Fast Address Sanity Checker.*" USENIX Annual Technical Conference (ATC), 2012. usenix.org/conference/atc12 — AddressSanitizer is the tool that identified the heap-buffer-overflow in the JSON parse loop bounds guard (§12.3) during the v0.53 audit pass.

[10] Gilley, S. T. *The Shape of Intent: Steering Artificial Intelligence in Cognitive Collaboration.* AIShell Labs LLC, Winston-Salem NC, 2026. Version 2.2. www.aishell.org — Develops the cognitive engineering framework for human–AI collaboration, including the Constraint-First pattern (§12), the Safety-First Systems architecture principles (§21), and the ethics of co-design (§24). Three principles from this work inform AIShell-Gate's design directly: that the architecture of safety must precede the architecture of capability; that delegating generation to AI is acceptable while delegating judgment is not; and that trust emerges from inspectability rather than from performance alone.

[11] OWASP Foundation. "*OWASP Top 10 for Large Language Model Applications.*" Version 2025. OWASP LLM AI Security & Governance Checklist. owasp.org/www-project-top-10-for-large-language-model-applications — Defines LLM06 (Excessive Agency): AI systems taking consequential actions beyond their intended scope. Model Sub-Delegation as defined in this paper is a more specific sub-case: an AI agent recruiting a second inference system to influence a policy decision governing the first agent's own execution session, distinguished from the broader Excessive Agency category by the inference-layer recruitment mechanism and the specific policy bypass intent.

Copyright © 2026 AIShell Labs LLC Winston-Salem NC USA. All Rights Reserved. — www.aishellgate.com · www.aishell.org

This document is proprietary to AIShell Labs LLC and is distributed subject to license. A valid license is required for production use of the software. See the License Notice in the software source for full terms.

This document is provided for informational purposes only and does not constitute legal, security, compliance, or operational advice. Shell commands and AI-driven automation carry inherent risk, including the possibility of irreversible data loss. You are solely responsible for evaluating the suitability of any approach described herein for your environment.

AIShell Labs LLC makes no warranty, express or implied, regarding the accuracy or completeness of this document. To the maximum extent permitted by applicable law, AIShell Labs LLC shall not be liable for any damages arising from reliance on this document or use of the software described herein.

Sean T. Gilley is the author of *The Shape of Intent: Steering Artificial Intelligence in Cognitive Collaboration*.

"AIShell-Gate" Copyright (c) 2026 AIShell Labs LLC Winston-Salem NC USA. All Rights Reserved. Author: Sean T. Gilley. Do not remove this notice. Use of this Software requires a valid license. Evaluation copies expire 30 days from download.

THIS SOFTWARE IS PROVIDED BY AISHELL LABS LLC "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL AISHELL LABS LLC BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

For full license terms see the LICENSE file or: www.aishellgate.com · www.aishell.org/aishellgate